TEMA 2 - Dart & Flutter

La decisión más importante: qué herramienta elegir

Una vez que hemos estudiado en el tema anterior “The state of the art” en relación con el desarrollo de aplicaciones multiplataforma, es el momento de realizar una elección que tendrá como consecuencia directa el aprendizaje de una tecnología específica, y como indirecta, el dejar en segundo plano el de otras alternativas.

La elección para el presente módulo ha sido **Dart/Flutter**, por diferentes motivos que pasamos a exponer:

- **Desarrollo multiplataforma:**
  Uno de los motivos principales, si no el que más, Flutter facilita la creación de aplicaciones que funcionan sin problemas en iOS y Android desde una única base de código

- **Rendimiento “casi” nativo:**
  Flutter utiliza el lenguaje Dart y un motor de renderizado propio para ofrecer un rendimiento similar al nativo, asegurando una experiencia de usuario fluida y receptiva. Este era el talón de Aquiles que tradicionalmente lastraba a las tecnologías no nativas, y que Flutter supera con notabilísimos resultados

- **Desarrollo rápido y eficiente:**
  La función de "hot reload" de Flutter permite a los desarrolladores ver los cambios en tiempo real, lo que acelera el proceso de desarrollo y reduce el tiempo de lanzamiento.

- **Gran comunidad y recursos:**
  Flutter cuenta con una comunidad activa y creciente, lo que se traduce en un fácil acceso a recursos, soporte y soluciones a problemas comunes. Tener a Google detrás del producto, tampoco resulta desdeñable.
  Flutter Málaga:

  https://www.meetup.com/es-ES/flutter-malaga/
  https://x.com/fluttermalaga?lang=es

- **Interfaz de usuario atractiva, personalizable y ligera:**
  Flutter ofrece una amplia gama de widgets y herramientas para crear interfaces de usuario atractivas y personalizadas, adaptándose a las necesidades de cada proyecto. Es, coloquialmente, como un kit de piezas que es necesario conocer para montar y obtener con ellas resultados personalizados. Visual Studio Code es

un IDE ligero que facilita mucho la eficiencia en la fase de desarrollo. En cualquier caso, sigue haciendo falta utilizar Android Studio para la emulación

- **Reducción de costos:**
  Al utilizar una base de código única para varias plataformas, Flutter ayuda a reducir los costos de desarrollo y mantenimiento.

- **Integración con funciones nativas:**
  Flutter permite acceder a las funciones y API nativas de los dispositivos, lo que mejora la experiencia del usuario y facilita la integración con otras aplicaciones

- **Adaptabilidad y futuro:**
  Flutter está diseñado para ser adaptable a futuras tecnologías y tendencias en el desarrollo móvil, asegurando que las aplicaciones están preparadas para los desafíos del mañana. En este sentido, se cuenta con un background y una perspectiva global de las que las tecnologías previas carecen. Aunque esto, el tiempo lo dirá



- **Menos mantenimiento:**
  Las aplicaciones de Flutter requieren menos mantenimiento a largo plazo, ya que solo necesitan actualizarse una vez en el código base compartido. Esto es característico de herramientas híbridas y multiplataforma.

Entre los inconvenientes más reseñables, el principal es el de tener que comenzar por aprender un lenguaje de programación nuevo: Dart, si bien su curva de aprendizaje no es muy empinada, sobre todo si se dominan los fundamentos de la POO.

En cualquier caso, esto representa un inconveniente sí y solo sí ya se domina previamente JavaScript, que nos llevaría más cómodamente hacia React o Ionic.

Se recomienda la lectura de este artículo para más información.

Introducción a DART



Dart es un lenguaje de programación Open Source lanzado por Google en 2011 con el objetivo de ser utilizado como un lenguaje tipado (y por tanto estable) que interactuara con el DOM, y compitiera directamente con JavaScript, siendo al mismo tiempo de propósito general.

En los años posteriores a su lanzamiento, JavaScript evoluciona de manera inesperada e incluso es progresivamente sustituido por TypeScript para darle una envoltura tipada. El desarrollo y uso de Flutter se detiene, y no es hasta el lanzamiento del SDK de Flutter en 2017, que no se revitaliza.

En la web oficial, es descrito como “lenguaje optimizado para UI”, puesto que con el mismo lenguaje se pueden describir tanto las vistas como la lógica de una aplicación, pero en realidad, es multifuncional.

En relación con el desarrollo de Apps, no es un lenguaje nativo, lo que significa que no está vinculado a un sistema operativo en particular, como Swift, que solo es comprendido por dispositivos iOS o Kotlin para Android (aunque debemos decir que, actualmente está disponible una nueva versión de Kotlin, denominada Kotlin Multiplatform que juega en la misma división que Flutter).

Para poder desarrollar con soltura una aplicación móvil en Flutter, será necesario conocer primero la sintaxis de Dart, que es bastante sencilla por otra parte, con una curva de aprendizaje bastante suave, y lo más importante de todo, dominar los principios de la orientación a objetos.

Dart es un lenguaje que materializa la orientación a objetos de forma bastante radical. En Dart, todo es un objeto, y precisamente este enfoque es lo que facilita la organización del código, la reutilización, y el diseño de aplicaciones complejas.



La POO en Dart, como no podría ser de otra forma, se basa en clases, objetos, herencia y polimorfismo, permitiendo una estructura clara y modular para el código.

Nos permite crear objetos que podemos personalizar y utilizar para construir nuestra aplicación. De hecho, Flutter en particular, se basa de manera fundamental en una amplísima base de objetos ya definidos aunque personalizables, los ubicuos widgets.

Finalmente, en el momento de la compilación, nuestro código en Dart será “traducido” para que pueda funcionar en la plataforma que nos interesa: iOS, Android o la web. También puede ser transpilado a Javascript.

Características Principales de Dart

- **Orientado a objetos:** Dart utiliza un modelo de objetos que es familiar para muchos desarrolladores, con clases, interfaces y herencia

- **Compilación Just-In-Time (JIT) y Ahead-Of-Time (AOT):** Dart puede compilar el código en tiempo de ejecución (JIT) para un desarrollo rápido y una iteración fluida. También admite la compilación AOT para producir código nativo rápido

- **Soporte para concurrencia:** A través de 'Isolates', Dart maneja la ejecución concurrente, permitiendo que las aplicaciones realicen múltiples tareas a la vez sin compartir memoria

- **Soporte para programación asíncrona:** permite ejecutar operaciones que tardan tiempo, sin bloquear el hilo principal de la aplicación, lo que es crucial para mantener la interfaz operativa

- **Sintaxis clara y simple:** Dart está diseñado para ser fácil de aprender y escribir, ofreciendo una sintaxis moderna y limpia

- **Null Safety:** Dart ayuda a prevenir errores comunes asegurando que los valores no nulos sean manejados correctamente

- **Ecosistema rico:** Dart viene con una biblioteca estándar sólida y un ecosistema de paquetes, lo que facilita la integración de una amplia gama de funcionalidades

- **Es un lenguaje versátil**, por lo que, aún habiéndose creado para el desarrollo de Apps, podría utilizarse sin problema en otro tipo de aplicaciones o situaciones: aplicaciones de escritorio, backend, sistemas empotrados...

Introducción a Flutter



Flutter, es un SDK (Software Development Kit) o framework de código abierto desarrollado por Google en 2017, que permite crear aplicaciones compiladas de forma nativa en múltiples plataformas utilizando un único código base y una multitud de widgets predefinidos que facilitan y uniformizan esta tarea.

En Flutter se pueden crear aplicaciones móviles, web y de escritorio, así como juegos 2D y 3D.

Y como ya sabemos, está construido sobre el lenguaje de programación Dart, diseñado ex-profeso para optimizar tanto el desarrollo como el rendimiento de las aplicaciones.



El framework cuenta con una amplia biblioteca de widgets (no nos cansamos de resaltarlo…) y elementos de interfaz de usuario prediseñados, lo que facilita la creación de aplicaciones visualmente atractivas, simplemente combinando y montando widgets. Flutter también admite programación asíncrona y concurrente, pues estas son características de Dart.

Versiones

Con la versión 1.0, lanzada oficialmente a finales de 2018, se podían desarrollar apps para Android y iOS con unos resultados bastante prometedores.. Pero Google no quería parar ahí y fue el 3 de marzo de 2021 cuando se lanzó oficialmente la versión 2.0 (Flutter 2).

Con Flutter 2, ya se podía usar la misma base de código para enviar aplicaciones nativas a cinco sistemas operativos: iOS, Android, Windows, macOS y Linux; así como experiencias web dirigidas a navegadores como Chrome, Firefox, Safari o Edge.

Flutter 2 incluso se puede integrar en automóviles, televisores y electrodomésticos inteligentes, llevando aún más allá las aplicaciones multiplataforma, al ámbito de IoT.

- La versión 3.0, lanzada en 2024 es capaz de dar soporte más estable a las plataformas soportadas previamente, y ofrece la posibilidad de desarrollar aplicaciones de escritorio más funcionales. Además, presenta las siguientes mejoras:

- La incorporación de Impeller: la versión Flutter 2.0 usaba Skia como principal motor gráfico. En cambio, Flutter 3.0 presenta Impeller, un nuevo motor de renderizado experimental diseñado para reemplazar a Skia en el futuro. Esta novedad, prevé mejorar el rendimiento gráfico.

- Firebase integrado: En la versión 2.0, los desarrolladores tenían que hacer más trabajo manual para configurar Firebase correctamente. En esta nueva versión, Flutter introduce una mejora en la integración para facilitar su uso.

- Material You (Material Design 3): La versión Flutter 2.0 incluía el soporte para Material Design, un sistema de diseño visual de Google. En cambio, ahora Flutter 3.0 integra soporte extendido para Material You, lo que permite que las aplicaciones adapten automáticamente sus temas de colores en función de las preferencias del sistema en dispositivos que sean Android 12 o superiores.

Inconvenientes

Dado que en la introducción de este tema, hemos hablado sobradamente de las razones positivas por las que vamos a utilizar Flutter, pasaremos a detallar sus inconvenientes, que haberlos, los hay:

- Dart no es tan popular a día de hoy como JavaScript, y es necesario invertir tiempo y esfuerzo en conocerlo antes de ponerse a desarrollar. Esto es relativo, pues la sintaxis es tan ·limpia" que no requiere demasiado esfuerzo

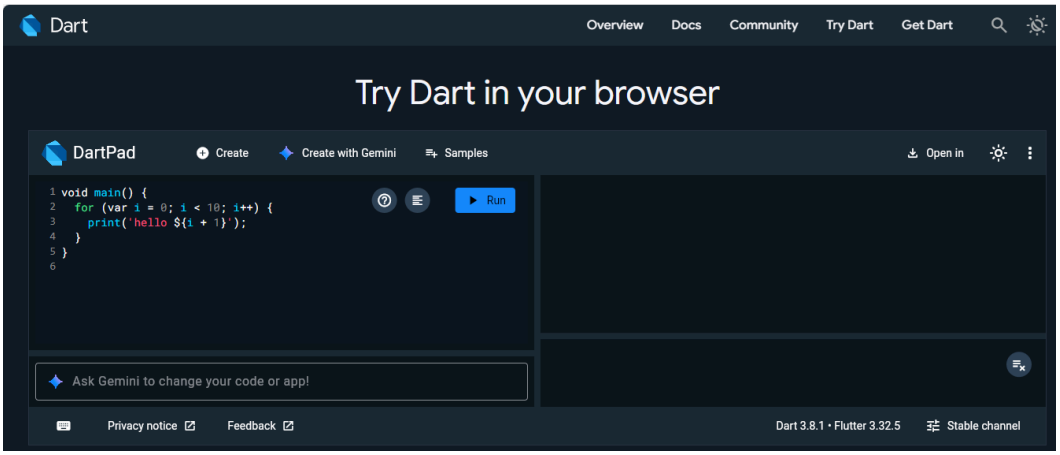
- Aunque está creciendo, el ecosistema de Flutter no es tan extenso como el nativo o algunas otras soluciones multiplataforma, debido, sobre todo, a su relativa juventud. Este inconveniente puede convertirse en ventaja, si nos enganchamos a un carro ganador (el tiempo lo dirá) de manera temprana, se nos abrirá un abanico de posibilidades laborales potencialmente interesantes

- Las aplicaciones creadas con Flutter pueden ser más grandes y consumir más memoria que las aplicaciones nativas, de código mucho más optimizado ya que las aplicaciones creadas con Flutter llevan incluido el motor Flutter (15-20 Mbytes). Solo es preocupante si se trata de aplicaciones muy sencillas, ya que el tamaño del código del motor será muy extenso comparativamente con la aplicación en sí

Comenzando a trabajar: Instalación de Dart y Flutter

Para empezar a trastear con Dart no sería necesario, en principio instalar nada. Podemos utilizar **DartPad** desde el navegador.

DartPad es una IDE en línea gratuita, desarrollada por Google, que permite escribir, ejecutar y compartir código Dart pero también Flutter (mucho más que un simple "hola mundo") directamente desde tu navegador web.



Es ideal para aprender el lenguaje Dart, probar pequeños fragmentos de código o compartir ejemplos, sin necesidad de instalar un entorno de desarrollo.

En este artículo tienes una explicación más detallada de las opciones de trabajo y configuración. Puede ser una buena forma de acercarse por primera vez al lenguaje Dart (aunque tb puede ejecutar el SDK de Flutter). Se recomienda probar los ejemplos.

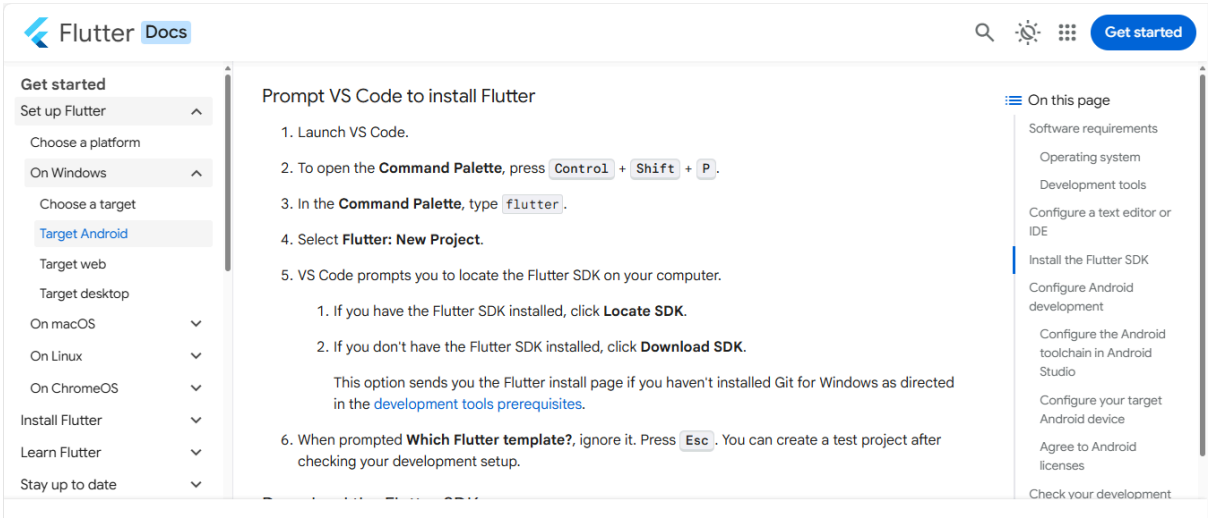
Instalando un IDE en local

Si visitamos la web oficial de Flutter, y nos vamos al apartado Install, nos detallarán el proceso completo para instalar el IDE a utilizar. Lo primero evidentemente sería elegir la plataforma (Windows, Linux, iOS, Chrome OS) y se podrá hacer de dos formas: a través de Visual Studio Code, desencadenando el proceso a partir de la instalación de plugins, o manualmente.



Podéis elegir la opción que queráis, pero ya que VSC va a ser nuestro IDE principal, haciéndolo desde aquí, aceleramos el proceso. Instalaremos de una vez Dart, Flutter y todas las ayudas que nos ofrece VSC para el desarrollo en estas tecnologías.

Un punto importante a destacar es que es muy recomendable, por no decir fundamental, utilizar git para realizar un control de versiones estricto, sobre todo en Flutter.



Para comprobar que todo ha ido correctamente, abrir una ventana cmd y escribir:

```
flutter -version
```

Es posible que se avise de la existencia de una nueva versión disponible, de ser así:

```
flutter upgrade
```

Para comprobar que todo esté actualizado (librerías, paquetes, etc…) podemos introducir el comando:

```
flutter doctor -v
```

En el momento de redactar este documento, la versión última de Flutter es la 3.35.1 y de Dart, la 3.9.0

Por ahora, con esta instalación podemos trabajar perfectamente en Dart y posteriormente en Flutter. Más adelante, tendremos que instalar también Android Studio, principalmente para poder ejecutar y probar aplicaciones Flutter en dispositivos Android, ya sean emuladores o físicos, y para tener acceso a las herramientas y dependencias necesarias para el desarrollo de aplicaciones Android.

Para emular dispositivos iOS, necesitaríamos instalar Xcode.





Es hora de aprender Dart. Llegados a este punto, estaremos preparados.